

From ML Lab to a Universal Interactive Learning Platform

Fundamental building blocks for software engineering, mathematics, physics, and beyond.



Executive summary

CORE INSIGHT

The reusable product is an **interactive exhibit engine** - not an ML course.

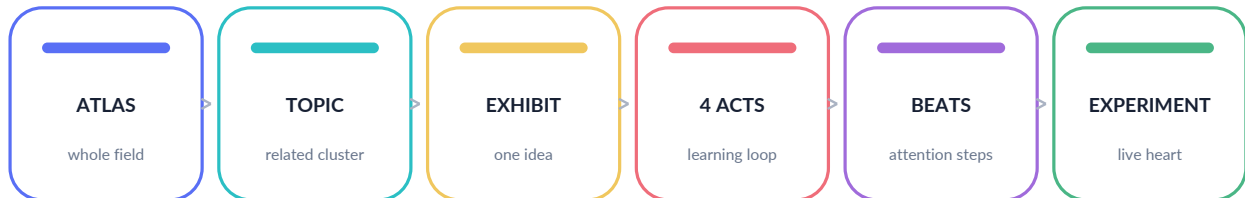
The subject matter can change. The learning choreography, experiment contract, and concept graph can remain shared.

What scales

- 01 Learning loop
- 02 Experiment contract
- 03 Concept graph

The reusable system at a glance

A strict hierarchy turns a broad field into small, authored, interactive experiences.



01 FOCUS

One exhibit, one question

Constrain scope so the learner can see and manipulate the central idea.

02 CAUSALITY

One action, visible effect

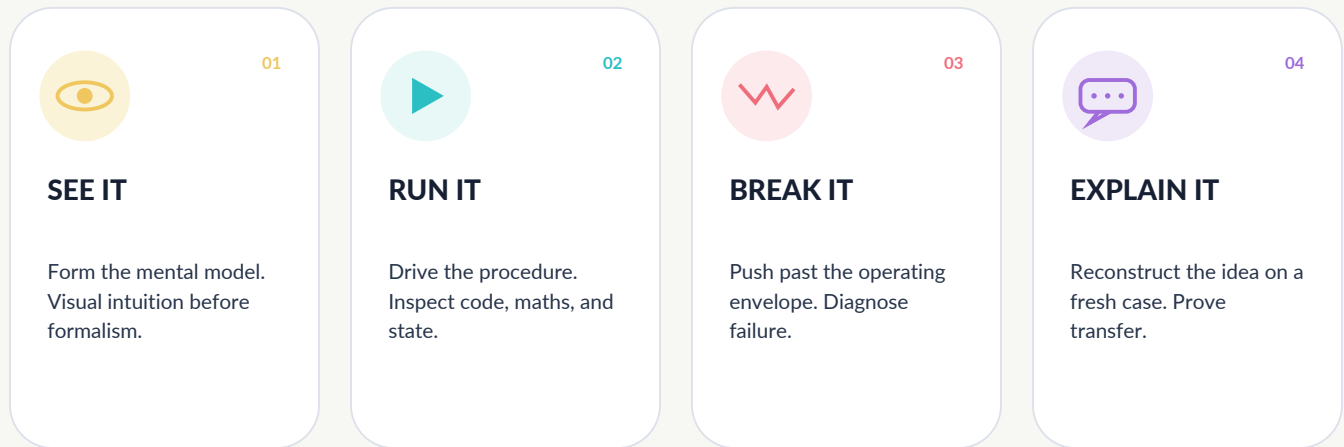
Every control should produce an immediate and measurable change.

03 TRANSFER

A fresh case proves learning

Completion is not a click. The learner reconstructs or predicts.

The four-pass learning loop



How the loop translates across domains

The same four learning moves can host very different knowledge objects and simulations.

ACT	SOFTWARE ENGINEERING	MATHEMATICS	PHYSICS
See it	Observe a trace, state change, or system behaviour.	Build geometric or numerical intuition.	Observe a phenomenon and linked quantities.
Run it	Step through code, messages, and state transitions.	Execute the derivation or algorithm.	Run the model and equations through time.
Break it	Inject concurrency, latency, load, or invalid state.	Find edge cases, singularities, and counterexamples.	Push parameters, assumptions, or numerical step size.
Explain it	Repair the design and justify the invariant.	Solve an unseen instance or prove a claim.	Predict a new configuration and reconcile representations.



Why "Break it" is the differentiator

Failure is promoted from a footnote to a first-class learning object. The learner discovers the operating envelope, diagnoses what failed, and sees which assumptions mattered.

The exhibit contract

01 One exhibit, one idea

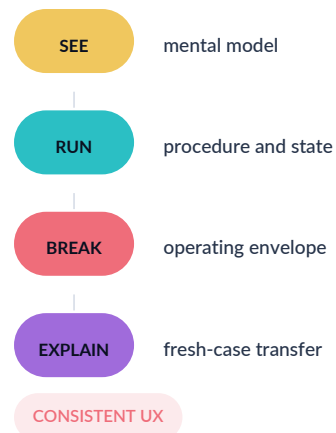
Each exhibit owns one conceptual question, one visible phenomenon, and one approximately twenty-minute outcome.

EXHIBIT
centralQuestion
twentyMinutePromise
buildsOn[] / leadsTo[]
compareWith[] / journeys[]
acts[] / experiment

SCOPE CONTROL

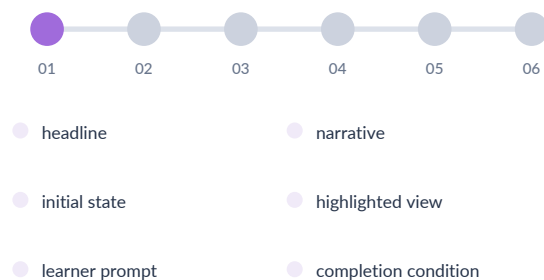
02 A fixed learning lifecycle

The four-pass loop is the stable spine. It gives every exhibit the same rhythm while leaving the subject matter free to change.



03 Beats shape attention

A beat introduces one claim, focuses one representation, and changes only part of the experiment. It is both the learner-facing step and the authoring unit.



04 A live causal experiment

The best exhibit gives the learner one dominant lever and makes the consequence immediate, visible, and measurable.



DESIGN RULE

One controlled cause -> an immediate, visible, and measurable effect.

Interaction and representation

05

A domain-independent interaction kit

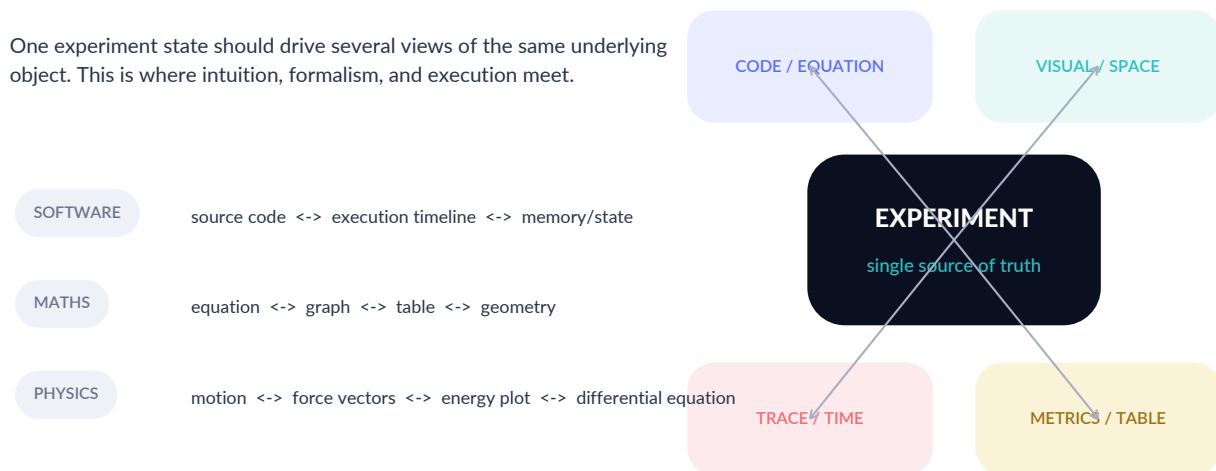
Most exhibits can be composed from a compact set of learner actions. The same controls can drive code, geometry, or simulation.



06

Synchronized representations

One experiment state should drive several views of the same underlying object. This is where intuition, formalism, and execution meet.



07

Instrument the invisible state

Readouts are not decoration. They surface the quantities the learner must reason about.



Learning architecture

08

Counterfactual and side-by-side comparison

Hold almost everything constant. Change one condition. The learner can then attribute the difference rather than merely notice it.

A / BASELINE



same model / condition off

B / CHANGED



same model / condition on

MUTEX OFF / ON

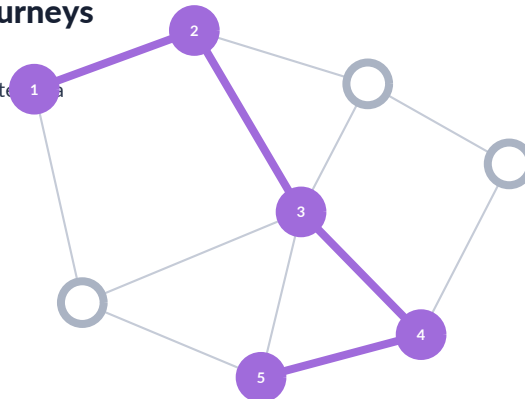
RAW / SCALED

DAMPED / UNDAMPED

09

A concept graph plus curated journeys

The graph represents knowledge. A journey is an editorial route to a particular audience or goal. One exhibit can appear in several journeys.



KNOWLEDGE GRAPH

CURATED JOURNEY

10

A consistent narrative shell

Consistency lets the learner spend attention on the subject rather than relearning the interface.

VISUAL THESIS

20-MINUTE PROMISE

METADATA + PREREQUISITES

FOUR-PASS NAVIGATOR

BEAT PROGRESS

NEXT CONCEPT

How the system travels across domains

The shell remains stable. The knowledge objects, renderer families, and failure cases change.

DOMAIN	CORE OBJECTS	BEST RENDERERS	PRODUCTIVE BREAK CASES	TRANSFER PROOF
Software engineering	Code, state, messages, queues, locks, caches, invariants.	Execution trace, graph/network, state machine, memory view.	Races, latency, retries, partial failure, invalid state, load.	Repair the design and justify the invariant and trade-offs.
Mathematics	Equations, transformations, geometric objects, proof steps, matrices.	Geometry, symbolic steps, tables, graphs, matrix views.	Edge cases, singularities, counterexamples, non-convergence.	Solve an unseen instance or prove a claim.
Physics	Particles, fields, forces, energy, momentum, time.	Vectors, particles, time plots, phase space, system diagrams.	Extreme parameters, violated assumptions, integration error.	Predict a new setup and reconcile the linked representations.

Three renderer families to prove first

01

EXECUTION TIMELINES

Code, messages, locks, queues, state transitions.

PRIMARY TEST

Can the same state drive code and trace?

02

MATHEMATICAL GEOMETRY

Graphs, transformations, matrices, symbolic steps.

PRIMARY TEST

Can geometry and notation stay synchronized?

03

CONTINUOUS-TIME SIMULATION

Particles, vectors, energy, phase, integration.

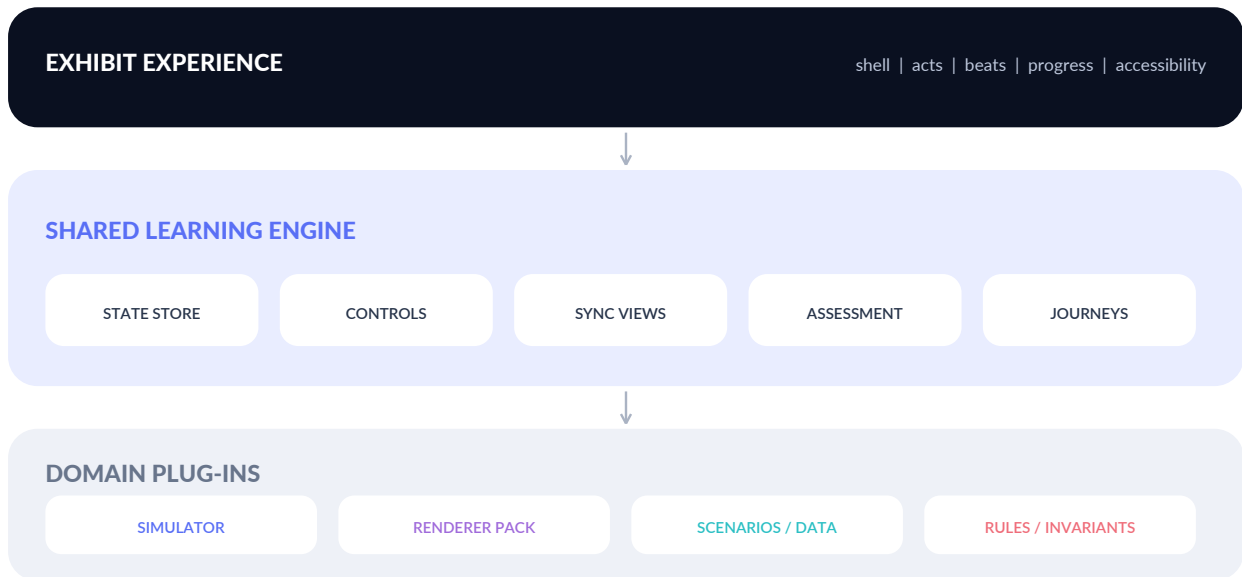
PRIMARY TEST

Can the simulation remain stable and inspectable?

What stays shared, what becomes pluggable

Recommended platform boundary

Shared learning infrastructure above; domain-specific simulation and rendering below.



EXPERIMENT API

```
Experiment {
  initialState
  update(action)
  derive(state)
  step(state, dt)
  reset()
  presets[]
  invariants[]
  views[]
}
```

The contract is intentionally small. The simulator owns domain logic; the shell owns learning flow and presentation.

Design rules

- Keep the content schema domain-neutral.
- Make simulator state deterministic and serializable.
- Store beat configuration as data, not layout code.
- Make every view subscribe to one source of truth.
- Treat failure presets as first-class content.
- Support reset, replay, and shareable experiment URLs.

Three pilot exhibits

SOFTWARE ENGINEERING

Race Conditions

PLATFORM CAPABILITY TESTED

Execution timelines + state machines

SEE

See identical operations produce different outcomes.

RUN

Step through thread interleavings and shared state.

BREAK

Inject delay, load, and partial failure.

EXPLAIN

Repair with a lock, atomic operation, or transaction; justify the invariant.

MATHEMATICS

Eigenvectors

PLATFORM CAPABILITY TESTED

Geometry + symbolic and matrix views

SEE

Watch a transformation move most vectors but preserve special directions.

RUN

Manipulate the matrix and trace the calculation.

BREAK

Explore repeated, complex, or defective cases.

EXPLAIN

Identify special directions for a new transformation and explain why.

PHYSICS

Harmonic Oscillation

PLATFORM CAPABILITY TESTED

Continuous simulation + linked plots

SEE

See position, velocity, and energy move together.

RUN

Run the differential equation through time.

BREAK

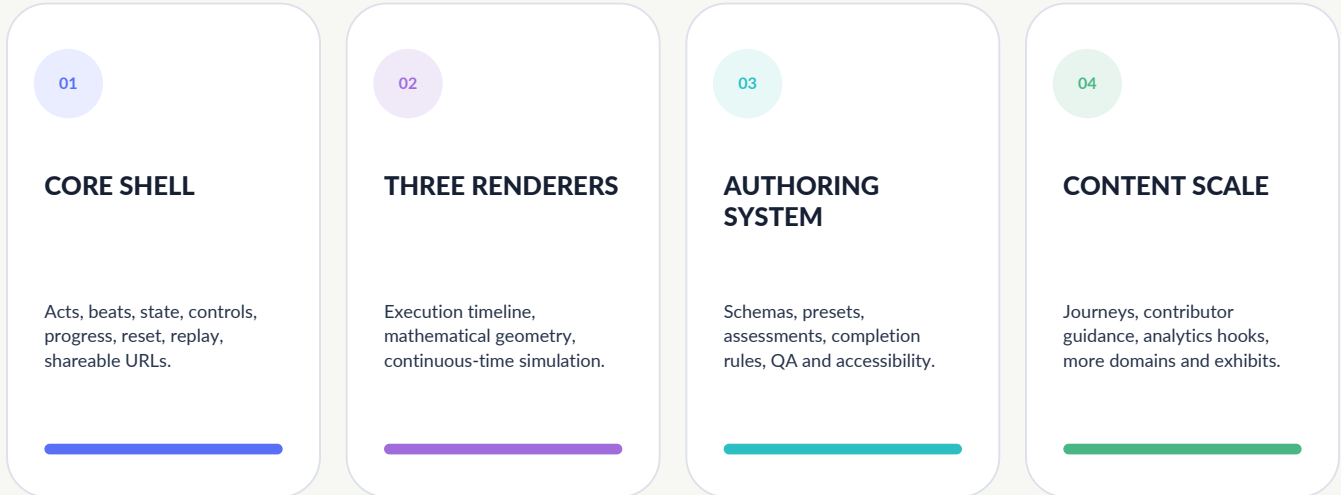
Alter damping, forcing, or step size until behaviour fails.

EXPLAIN

Predict phase and energy exchange in a new setup.

Build the engine, prove the pattern, then scale

A four-phase sequence reduces architecture risk before content volume increases.



Decision gates

GATE	EVIDENCE OF SUCCESS	FAILURE SIGNAL
Reuse	The same shell hosts all three pilots.	Each domain requires a custom page architecture.
Causality	One control produces a visible and measurable effect.	The learner cannot say what changed or why.
Transfer	Explain it uses a genuinely fresh case.	Completion is equivalent to clicking Next.
Authorability	A new exhibit is mostly data plus plug-ins.	Every exhibit becomes bespoke application work.

DECISIVE TEST

Can Race Conditions, Eigenvectors, and Harmonic Oscillation use the same exhibit shell, four-pass runner, beat model, controls, readouts, assessments, and journey graph - while replacing only the simulator and renderer?

Reusable component inventory



Shared engine

EXPERIENCE

- ☒ Exhibit shell + act/beat navigation
- ☒ Progress persistence
- ☒ Audio, keyboard, accessibility
- ☒ Reset, replay, presets, shareable URLs

LEARNING

- ☒ Prediction before reveal
- ☒ Prompts, checkpoints, completion
- ☒ Compare mode + failure presets
- ☒ Transfer tasks + next-concept links

PLATFORM

- ☒ State manager + renderer registry
- ☒ Instrumentation/readouts
- ☒ Concept graph + journeys
- ☒ Analytics + authoring hooks



Domain plug-ins

SIMULATOR

The executable model and state transitions.

RENDERER PACK

Domain views: timelines, geometry, particles, equations, graphs.

SCENARIOS / DATA

Starting cases, presets, and pathological examples.

VOCABULARY

Labels and explanations appropriate to the subject.

INVARIANTS

What must remain true and what counts as a failure.

ASSESSMENT RULES

What the learner must predict, repair, or explain.

Evidence base

Patterns were derived from the live ML Lab site and representative exhibits, reviewed on 25 June 2026.

- **ML Lab homepage** atlas, four-pass model, journey
- **Linear Regression** metadata, live fit, beats
- **Classification** threshold, readouts, predict-first
- **Gradient Descent + Feature Scaling** playback, scrubbing, operating envelope
- **Overfitting + Data Leakage** counterfactual comparison and honest evaluation

Build the shell once. Prove it across three domains. Then scale the atlas.